

LAB ASSIGNMENT 2



DESIGN AND ANALYSIS OF ALGORITHMS LAB

(Course Code: ENCA351)

Algorithm Performance Measurement and Benchmarking Tool

NAME : Tamanna Deshwal

Roll No. : 2401201138

Course : BCA (AI & DS)

SEMESTER : V

SESSION : "A"

SUBMITTED TO : Dr. Aarti

1. Aim of the Lab

To develop an Algorithm Performance Measurement and Benchmarking Tool in Python that executes algorithms and code snippets for different input sizes, measures their execution time, memory usage and number of operations, compares theoretical and experimental complexity, and shows the results as tables and graphs.

Objectives

- Understand practical techniques for algorithm performance analysis.
- Measure execution time and memory consumption of programs.
- Compare theoretical complexity with experimental results.
- Analyse the effect of input size on algorithm behaviour.
- Develop an interactive benchmarking application using Streamlit.

2. Introduction

2.1 Importance of algorithm analysis

Algorithm analysis tells us how the running time and memory needed by a program grow when the input becomes larger. An algorithm that works quickly on 100 items may become very slow on 1,00,000 items. By studying notations such as $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$, a programmer can choose the right algorithm before using it in a real application.

2.2 Need for performance measurement

Theory only gives the growth trend, not the real numbers. By actually running the code and measuring execution time and memory, we can check whether the practical behaviour matches the theory and we can discover hidden costs such as the call stack in recursion or the cost of big-number multiplication in factorial programs.

2.3 What this project does

The project has one reusable BenchmarkEngine that measures any function. It is used on six programs: Linear Search, Binary Search, Single Loop Traversal, Nested Loop Traversal, Recursive Factorial and Iterative Factorial. A Streamlit web application (app.py) and a Jupyter notebook (project_notebook.ipynb) present the results.

3. Project Setup and Tools

3.1 Environment setup

The project was set up with a Python virtual environment so that all libraries stay inside the project folder. The following commands were used:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

```
streamlit run app.py
```

3.2 Libraries used

Library	Use in the project
streamlit	Creates the interactive web application (app.py)
pandas	Stores benchmark results in a DataFrame / CSV file
matplotlib	Draws the comparison graphs
numpy	Calculates the growth exponent (log-log slope)
memory_profiler	Installed as required by the lab (memory is measured with tracemalloc, which is built into Python)
jupyter	Runs project_notebook.ipynb
time, tracemalloc (built-in)	time.perf_counter() measures time, tracemalloc measures peak memory

3.3 Project files

File	Purpose
search_analysis.py	Linear Search, Binary Search and the dataset generator (counts comparisons)
code_benchmark.py	Single loop, nested loop, recursive factorial, iterative factorial (count operations)
benchmark_engine.py	BenchmarkEngine class: measures time, memory and operations; runs all benchmarks
visualization.py	Draws the graphs and builds the theoretical vs observed complexity table
app.py	Streamlit web application with 4 tabs
project_notebook.ipynb	The complete project executed step by step in Jupyter
README.md, requirements.txt, .gitignore	Documentation, library list and git ignore rules

3.4 How the measurement works

- **Execution time:** time.perf_counter() is called before and after the function. The run is repeated 5 times and the average is taken, because very small times change a little from run to run.
- **Memory usage:** tracemalloc.start() begins tracking and tracemalloc.get_traced_memory() returns the peak memory (in KB) used during one run. Memory is measured in a separate run because tracemalloc makes the code slower.
- **Number of operations:** Every function keeps a counter and returns (result, operations). For searches this counter is the number of comparisons. This count does not depend on the computer, so it is the most reliable proof of complexity.
- **Worst case for searches:** In the automated benchmark the key is not present in the list (-1), so both searches do the maximum work.

4. Theory of the Algorithms Used

4.1 Linear Search

Linear Search checks every element one by one from the start until the target is found or the list ends. It works on unsorted data.

4.2 Binary Search

Binary Search works only on a sorted list. It compares the target with the middle element and throws away half of the list in every step, so the interval keeps halving.

4.3 Single Loop Traversal

One loop visits each of the n elements exactly once, so it does n steps.

4.4 Nested Loop Traversal

A loop inside a loop visits every pair (i, j) . The outer loop runs n times and each time the inner loop runs n times, so $n \times n = n^2$ steps.

4.5 Recursive Factorial

Uses $n! = n \times (n-1)!$. The function calls itself n times, and every call stays on the call stack until the last one returns, so it needs $O(n)$ extra space.

4.6 Iterative Factorial

Uses one loop and one variable (result), multiplying 2, 3, ..., n . It needs only $O(1)$ extra space.

4.7 Theoretical complexity table

Algorithm / Pattern	Best Case	Average Case	Worst Case	Space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Single Loop	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Nested Loop	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Factorial (Recursive)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Factorial (Iterative)	$O(n)$	$O(n)$	$O(n)$	$O(1)$

5. Source Code

All the code written for this project is given below with a short explanation of each file.

5.1 search_analysis.py (Linear Search, Binary Search, dataset generator)

- **generate_dataset(size, sorted_data):** creates a list of unique random numbers. When sorted_data is True the list is sorted, which Binary Search needs.
- **linear_search(arr, target):** goes through the list from left to right. A counter "comparisons" is increased at every check. It returns (index, comparisons), or (-1, comparisons) when the target is absent.
- **binary_search(arr, target):** keeps two pointers low and high. In each step it checks the middle element: if equal it stops, if the middle is smaller it moves low up, otherwise it moves high down. The counter records the comparisons.

```
"""Task 2 - Search algorithms (Linear Search and Binary Search).

Both functions return (index, comparisons) so we can show the
number of comparisons made.
"""
import random

def generate_dataset(size, sorted_data=False):
    """Make a list of `size` unique random numbers."""
    data = random.sample(range(size * 10), size)
    if sorted_data:
        data.sort()
    return data

def linear_search(arr, target):
    """Returns (index of target or -1, number of comparisons)."""
    comparisons = 0
    for i, value in enumerate(arr):
        comparisons += 1
        if value == target:
            return i, comparisons
    return -1, comparisons

def binary_search(arr, target):
    """arr MUST be sorted. Returns (index of target or -1, comparisons)."""
    comparisons = 0
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        comparisons += 1
        if arr[mid] == target:
            return mid, comparisons
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1, comparisons

if __name__ == "__main__":
```

```
data = generate_dataset(10, sorted_data=True)
print("Data:", data)
print("Linear search for", data[3], "->", linear_search(data, data[3]))
print("Binary search for", data[3], "->", binary_search(data, data[3]))
```

5.2 code_benchmark.py (loops and factorial programs)

- **single_loop(n)**: adds numbers 0 to $n-1$ with one loop and counts the steps. Time $O(n)$.
- **nested_loop(n)**: two loops, adds 1 for every (i, j) pair, so the operation count is exactly n^2 . Time $O(n^2)$.
- **factorial_recursive(n)**: calls itself with $n-1$ until $n \leq 1$. It returns the value and the number of calls. Space $O(n)$ because of the call stack.
- **factorial_iterative(n)**: multiplies result by 2, 3, ..., n in a loop. Space $O(1)$.
- **SNIPPETS dictionary**: maps the names shown in the app to the functions, so the app can run any selected snippet.

```
"""Task 3 - Code snippets to benchmark.

Every function returns (result, operations) where `operations`
is a simple count of how many basic steps were done.
"""

def single_loop(n):
    """Sums 0..n-1 using a single loop.  $O(n)$ """
    total = 0
    operations = 0
    for i in range(n):
        total += i
        operations += 1
    return total, operations

def nested_loop(n):
    """Counts all  $(i, j)$  pairs using a nested loop.  $O(n^2)$ """
    total = 0
    operations = 0
    for i in range(n):
        for j in range(n):
            total += 1
            operations += 1
    return total, operations

def factorial_recursive(n):
    """ $n!$  using recursion.  $O(n)$  time,  $O(n)$  stack space."""
    if n <= 1:
        return 1, 1
    value, ops = factorial_recursive(n - 1)
    return n * value, ops + 1

def factorial_iterative(n):
    """ $n!$  using a loop.  $O(n)$  time,  $O(1)$  space."""
    result = 1
    operations = 0
    for i in range(2, n + 1):
        result *= i
        operations += 1
    return result, operations

# Names shown in the app -> function
```

```
SNIPPETS = {
    "Single Loop Traversal": single_loop,
    "Nested Loop Traversal": nested_loop,
    "Recursive Factorial": factorial_recursive,
    "Iterative Factorial": factorial_iterative,
}

if __name__ == "__main__":
    for name, func in SNIPPETS.items():
        print(name, "(n=10) ->", func(10))
```

5.3 benchmark_engine.py (Automated Benchmarking Engine)

- **measure(func, args, repeats):** runs the function once under tracemalloc to get peak memory, then runs it several times with time.perf_counter() to get the average time. It also reads the operation count returned by the function.
- **run(func, input_sizes, name, arg_builder):** repeats measure() for every input size and stores one result row (algorithm, input_size, time_sec, memory_kb, operations). arg_builder creates the input for a given n, for example a list of n numbers.
- **as_dataframe():** converts all results into a pandas DataFrame.
- **run_all_benchmarks():** runs all six programs with the suggested input sizes and returns the complete table. sys.setrecursionlimit(5000) is used so recursive factorial works for n = 1000.

```
"""Task 1 + 4 - Reusable benchmarking engine.

Measures execution time (time.perf_counter), peak memory (tracemalloc)
and number of operations for any function and many input sizes.
"""
import sys
import time
import tracemalloc

import pandas as pd

from search_analysis import linear_search, binary_search, generate_dataset
from code_benchmark import single_loop, nested_loop, factorial_recursive, factorial_iterative

# Recursive factorial with n = 1000 needs a deeper stack than the default
sys.setrecursionlimit(5000)

class BenchmarkEngine:
    def __init__(self):
        self.results = []

    def measure(self, func, args, repeats=5):
        """Run func once for memory and `repeats` times for time.
        Returns (avg_time_sec, peak_memory_kb, operations)."""
        # 1) memory (separate run, because tracemalloc slows code down)
        tracemalloc.start()
        output = func(*args)
        peak = tracemalloc.get_traced_memory()[1]
        tracemalloc.stop()

        # 2) time (average of several runs)
        total = 0
        for _ in range(repeats):
            start = time.perf_counter()
            func(*args)
            total += time.perf_counter() - start

        # functions return (result, operations)
        operations = output[1] if isinstance(output, tuple) else None
        return total / repeats, peak / 1024, operations

    def run(self, func, input_sizes, name=None, arg_builder=lambda n: (n,), repeats=5):
        """Benchmark func for every n in input_sizes."""
        name = name or func.__name__
        for n in input_sizes:
```

```

        t, mem, ops = self.measure(func, arg_builder(n), repeats)
        self.results.append({
            "algorithm": name,
            "input_size": n,
            "time_sec": t,
            "memory_kb": mem,
            "operations": ops,
        })
    return self

def as_dataframe(self):
    return pd.DataFrame(self.results)

# ---- Input sizes used by the automated run (Task 4) ----
SEARCH_SIZES = [100, 1000, 10000, 50000]
FACTORIAL_SIZES = [100, 500, 1000]
LOOP_SIZES = [100, 1000, 10000, 50000]
NESTED_SIZES = [100, 1000, 5000, 10000] # 50,000 would need 2.5 billion steps

def run_all_benchmarks():
    """Runs every algorithm for every input size and returns a DataFrame."""
    engine = BenchmarkEngine()

    # Worst case for both searches: the key is NOT in the list
    engine.run(linear_search, SEARCH_SIZES, name="Linear Search",
               arg_builder=lambda n: (generate_dataset(n), -1))
    engine.run(binary_search, SEARCH_SIZES, name="Binary Search",
               arg_builder=lambda n: (generate_dataset(n, sorted_data=True), -1))

    engine.run(factorial_recursive, FACTORIAL_SIZES, name="Recursive Factorial")
    engine.run(factorial_iterative, FACTORIAL_SIZES, name="Iterative Factorial")

    engine.run(single_loop, LOOP_SIZES, name="Single Loop")
    engine.run(nested_loop, NESTED_SIZES, name="Nested Loop", repeats=1)
    return engine.as_dataframe()

if __name__ == "__main__":
    df = run_all_benchmarks()
    print(df)
    df.to_csv("benchmark_results.csv", index=False)
    print("Saved benchmark_results.csv")

```

5.4 visualization.py (graphs and complexity table)

- **compare_chart():** draws a line graph of input size against time (or memory) for the algorithms given.
- **make_all_charts():** creates the 3 time graphs and 3 memory graphs and saves them as PNG files.
- **growth_exponent():** finds the slope of $\log(\text{time})$ against $\log(n)$ using numpy. A slope near 1 means linear growth, near 2 means quadratic growth and near 0 means almost constant.
- **complexity_table():** builds the table that places theoretical complexity next to the observed time, memory, growth exponent and trend.

```
"""Task 5 + 6 - Charts and complexity table."""
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")          # lets us save charts without opening a window
import matplotlib.pyplot as plt

# Theoretical complexity (Task 6)
THEORY = {
    "Linear Search": ("O(n)", "O(1)"),
    "Binary Search": ("O(log n)", "O(1)"),
    "Recursive Factorial": ("O(n)", "O(n)"),
    "Iterative Factorial": ("O(n)", "O(1)"),
    "Single Loop": ("O(n)", "O(1)"),
    "Nested Loop": ("O(n^2)", "O(1)"),
}

def compare_chart(df, algos, x_label, title, column="time_sec", y_label="Execution Time
(seconds)", filename=None):
    """Line chart comparing the given algorithms on one graph."""
    fig, ax = plt.subplots(figsize=(8, 5))
    for algo in algos:
        part = df[df["algorithm"] == algo]
        if len(part) > 0:
            ax.plot(part["input_size"], part[column], marker="o", label=algo)
    ax.set_xlabel(x_label)
    ax.set_ylabel(y_label)
    ax.set_title(title)
    ax.legend()
    ax.grid(True)
    if filename:
        fig.savefig(filename, dpi=150, bbox_inches="tight")
    return fig

def make_all_charts(df, folder="."):
    """Creates the 3 time charts + 3 memory charts. Returns list of file names."""
    groups = [
        ("search", ["Linear Search", "Binary Search"], "Dataset Size", "Linear Search vs Binary
Search"),
        ("factorial", ["Recursive Factorial", "Iterative Factorial"], "Input Size (n)",
"Recursive vs Iterative Factorial"),
        ("loops", ["Single Loop", "Nested Loop"], "Number of Iterations (n)", "Single Loop vs
Nested Loop"),
    ]
    files = []
    for key, algos, xlabel, title in groups:
        f1 = f"{folder}/graph_{key}_time.png"
        compare_chart(df, algos, xlabel, title + " - Time", filename=f1)
```

```

        f2 = f"{folder}/graph_{key}_memory.png"
        compare_chart(df, algos, xlabel, title + " - Memory", "memory_kb", "Peak Memory (KB)",
f2)
        files += [f1, f2]
        plt.close("all")
    return files

def growth_exponent(sizes, times):
    """Slope of log(time) vs log(n). ~1 = linear, ~2 = quadratic, ~0 = very flat."""
    slope = np.polyfit(np.log(sizes), np.log(times), 1)[0]
    return slope

def trend_name(exponent):
    if exponent < 0.4:
        return "Almost constant / logarithmic"
    if exponent < 1.6:
        return "Linear growth"
    return "Quadratic growth"

def complexity_table(df):
    """Theoretical vs observed table (Task 6)."""
    rows = []
    for algo, (time_c, space_c) in THEORY.items():
        part = df[df["algorithm"] == algo]
        if len(part) < 2:
            continue
        exp = growth_exponent(part["input_size"], part["time_sec"])
        biggest = part.iloc[-1]
        rows.append({
            "Algorithm": algo,
            "Time Complexity": time_c,
            "Space Complexity": space_c,
            "Largest n": int(biggest["input_size"]),
            "Time at largest n (s)": round(biggest["time_sec"], 6),
            "Memory at largest n (KB)": round(biggest["memory_kb"], 2),
            "Growth exponent": round(exp, 2),
            "Observed trend": trend_name(exp),
        })
    return pd.DataFrame(rows)

if __name__ == "__main__":
    df = pd.read_csv("benchmark_results.csv")
    print(make_all_charts(df))
    print(complexity_table(df).to_string(index=False))

```

5.5 app.py (Streamlit application)

The application has four tabs: (1) Search Analysis, (2) Code Benchmark, (3) Automated Benchmark with graphs, and (4) Complexity Analysis. It imports the other four files and displays their results as tables and charts.

```
"""Streamlit app - Algorithm Performance Measurement and Benchmarking Tool.
Run with: streamlit run app.py
"""
import random

import pandas as pd
import streamlit as st

from benchmark_engine import BenchmarkEngine, run_all_benchmarks
from search_analysis import linear_search, binary_search, generate_dataset
from code_benchmark import SNIPPETS
from visualization import compare_chart, complexity_table

st.set_page_config(page_title="Algorithm Performance Tool", layout="wide")
st.title("Algorithm Performance Measurement and Benchmarking Tool")

tab1, tab2, tab3, tab4 = st.tabs([
    "1. Search Analysis", "2. Code Benchmark", "3. Automated Benchmark", "4. Complexity
Analysis"])

# -----
# Tab 1 - Search algorithms (Task 2)
# -----
with tab1:
    st.header("Search Algorithm Analysis")
    size = st.number_input("Dataset size", 10, 200000, 10000, step=1000)
    key_choice = st.radio("Search key", ["Random element from the data", "Key not in data",
"Enter my own"])
    own_key = st.number_input("Your key", value=0, step=1)

    if st.button("Run Search"):
        data = generate_dataset(size, sorted_data=True)          # sorted so both searches work
        if key_choice == "Random element from the data":
            key = random.choice(data)
        elif key_choice == "Key not in data":
            key = -1
        else:
            key = int(own_key)
        st.write(f"Searching for **{key}** in {size} elements")

        engine = BenchmarkEngine()
        rows = []
        for name, func in [("Linear Search", linear_search), ("Binary Search", binary_search)]:
            t, mem, comps = engine.measure(func, (data, key))
            index = func(data, key)[0]
            rows.append({
                "Algorithm": name,
                "Result": f"Found at index {index}" if index != -1 else "Not found",
                "Comparisons": comps,
                "Time (s)": f"{t:.8f}",
                "Memory (KB)": round(mem, 3),
            })
        st.table(pd.DataFrame(rows))

# -----
# Tab 2 - Code snippets (Task 3)
# -----
with tab2:
    st.header("Code Benchmarking")
```

```

    chosen = st.multiselect("Select code snippets", list(SNIPPETS.keys()),
default=list(SNIPPETS.keys()))
    n = st.number_input("Input size n", 1, 5000, 500, step=100,
                        help="Kept below 5000 because nested loop is O(n^2) and recursion has a
depth limit")

    if st.button("Run Snippets"):
        engine = BenchmarkEngine()
        rows = []
        for name in chosen:
            t, mem, ops = engine.measure(SNIPPETS[name], (n,))
            rows.append({"Snippet": name, "Time (s)": f"{t:.8f}", "Memory (KB)": round(mem, 3),
"Operations": ops})
        st.subheader("Comparison table")
        st.table(pd.DataFrame(rows))

# -----
# Tab 3 - Automated benchmark + charts (Task 4 + 5)
# -----
with tab3:
    st.header("Automated Benchmarking")
    st.info("Runs every algorithm on all suggested input sizes. The nested loop with n = 10,000
takes a few seconds.")
    if st.button("Run Full Benchmark"):
        with st.spinner("Benchmarking... please wait"):
            st.session_state["df"] = run_all_benchmarks()

    if "df" in st.session_state:
        df = st.session_state["df"]
        st.dataframe(df)
        st.download_button("Download CSV", df.to_csv(index=False), "benchmark_results.csv")

    col1, col2 = st.columns(2)
    with col1:
        st.pyplot(compare_chart(df, ["Linear Search", "Binary Search"], "Dataset Size",
"Search: Size vs Time"))
        st.pyplot(compare_chart(df, ["Recursive Factorial", "Iterative Factorial"], "n",
"Factorial: n vs Time"))
        st.pyplot(compare_chart(df, ["Single Loop", "Nested Loop"], "Iterations", "Loops:
Iterations vs Time"))
    with col2:
        st.pyplot(compare_chart(df, ["Linear Search", "Binary Search"], "Dataset Size",
"Search: Memory",
                                "memory_kb", "Peak Memory (KB)"))
        st.pyplot(compare_chart(df, ["Recursive Factorial", "Iterative Factorial"], "n",
"Factorial: Memory",
                                "memory_kb", "Peak Memory (KB)"))
        st.pyplot(compare_chart(df, ["Single Loop", "Nested Loop"], "Iterations", "Loops:
Memory",
                                "memory_kb", "Peak Memory (KB)"))

# -----
# Tab 4 - Complexity table (Task 6)
# -----
with tab4:
    st.header("Theoretical vs Observed Complexity")
    if "df" in st.session_state:
        st.table(complexity_table(st.session_state["df"]))
        st.caption("Growth exponent = slope of log(time) vs log(n): about 1 means linear, about
2 means quadratic.")
    else:
        st.warning("Run the full benchmark in tab 3 first.")

```

6. Output Screenshots and Explanation

The application was run with the command `streamlit run app.py`. Each screenshot below is explained one by one.

6.1 Search Analysis (Task 2)

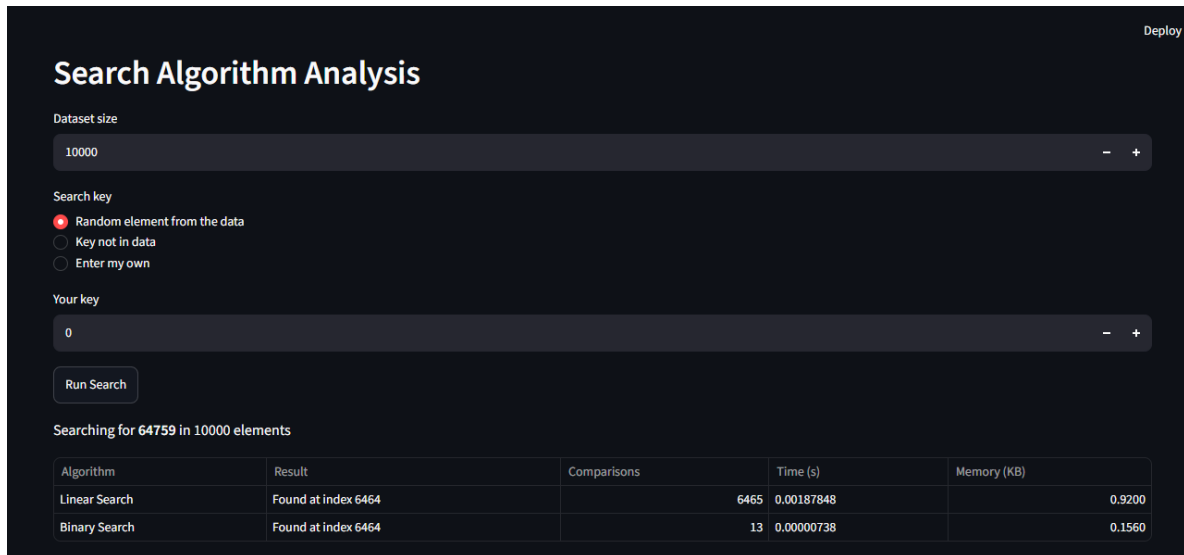


Figure 1: Search Analysis tab - Linear Search vs Binary Search

In this screenshot a list of 10,000 numbers was made and one number (64759) was searched. Both searches found it at index 6464. Linear Search took 6465 comparisons and 0.0019 seconds, but Binary Search took only 13 comparisons and 0.0000074 seconds. So Binary Search is much faster.

6.2 Code Benchmarking (Task 3)

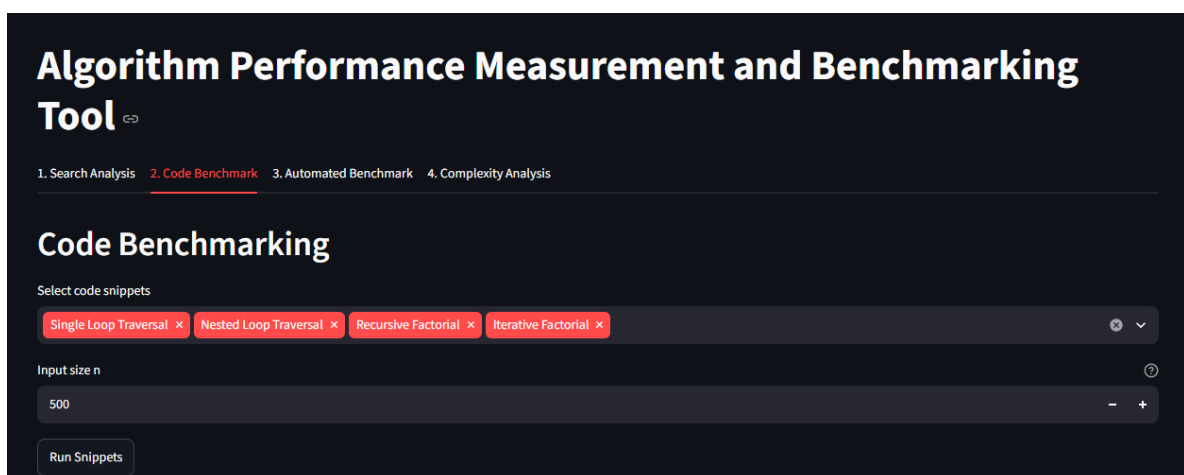


Figure 2: Code Benchmark tab - selecting snippets and input size

Here we choose which code snippets to run and the value of n . All four snippets are selected with $n = 500$.

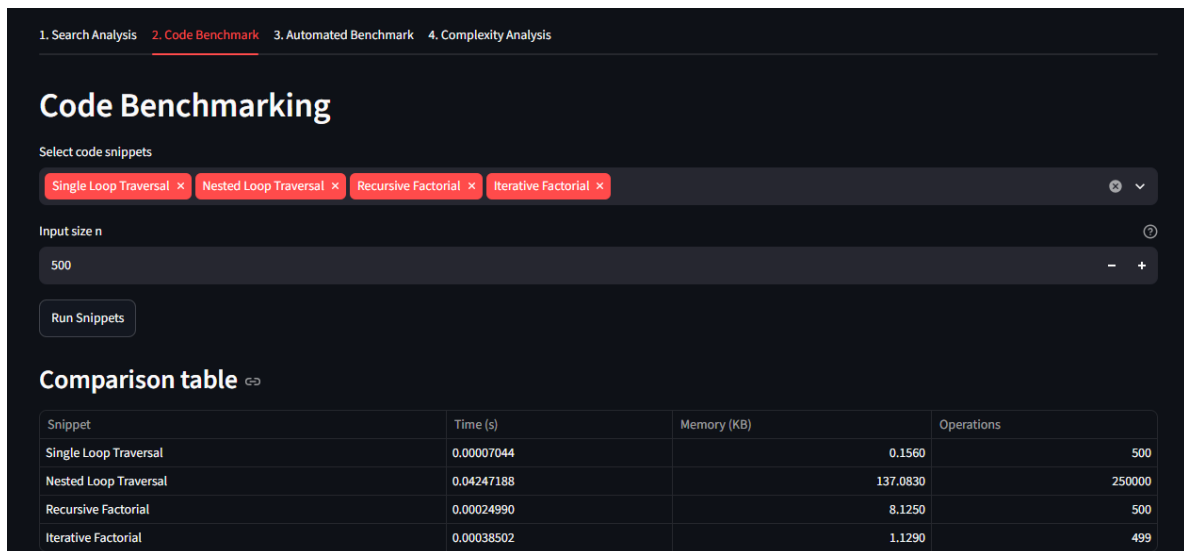


Figure 3: Comparison table of the code snippets for $n = 500$

The table shows time, memory and operations of each snippet. The nested loop took the most time (250000 operations) while the single loop took very little (500 operations). Recursive factorial used more memory (8.125 KB) than iterative factorial (1.129 KB).

6.3 Automated Benchmark (Task 4)

algorithm	input_size	time_sec	memory_kb	operations	Download as CSV
0 Linear Search	100	0.00001	252.667	100	
1 Linear Search	1000	0.0002	0.207	1000	
2 Linear Search	10000	0.0399	2.458	10000	
3 Linear Search	50000	0.0433	199.3428	50000	
4 Binary Search	100	0.000005	0	6	
5 Binary Search	1000	0.000008	0.0938	9	
6 Binary Search	10000	0.000006	0.125	13	
7 Binary Search	50000	0.000004	0.125	15	
8 Recursive Factorial	100	0.00008	0.1953	100	
9 Recursive Factorial	500	0.0006	8.125	500	
10 Recursive Factorial	1000	0.002	25.1123	1000	
11 Iterative Factorial	100	0.00004	0.2266	99	
12 Iterative Factorial	500	0.0004	1.1289	499	
13 Iterative Factorial	1000	0.0075	2.3672	999	
14 Single Loop	100	0.000007	0.0938	100	
15 Single Loop	1000	0.0001	0.1563	1000	

Figure 4: Automated benchmark results table (rows 0 to 15)

	algorithm	input_size	time_sec	memory_kb	operations	Download as CSV
6	Binary Search	10000	0.000006	0.125	13	
7	Binary Search	50000	0.000004	0.125	15	
8	Recursive Factorial	100	0.00008	0.1953	100	
9	Recursive Factorial	500	0.0006	8.125	500	
10	Recursive Factorial	1000	0.002	25.1123	1000	
11	Iterative Factorial	100	0.00004	0.2266	99	
12	Iterative Factorial	500	0.0004	1.1289	499	
13	Iterative Factorial	1000	0.0075	2.3672	999	
14	Single Loop	100	0.000007	0.0938	100	
15	Single Loop	1000	0.0001	0.1563	1000	
16	Single Loop	10000	0.0024	6.5986	10000	
17	Single Loop	50000	0.0183	12.4854	50000	
18	Nested Loop	100	0.0332	2.79	10000	
19	Nested Loop	1000	0.8856	301.165	1000000	
20	Nested Loop	5000	48.5091	0	25000000	
21	Nested Loop	10000	34.6158	0	100000000	

Figure 5: Automated benchmark results table (rows 6 to 21)

This table is the result of the automated benchmark. It runs every program for all input sizes and saves time, memory and operations. Both screenshots show the same table, one at the top and one at the bottom.

The operations column matches the theory: linear search and single loop did n steps, binary search did only 6 to 15 steps, and the nested loop did $n \times n$ steps (for example 100000000 for $n = 10000$).

6.4 Performance graphs (Task 5)

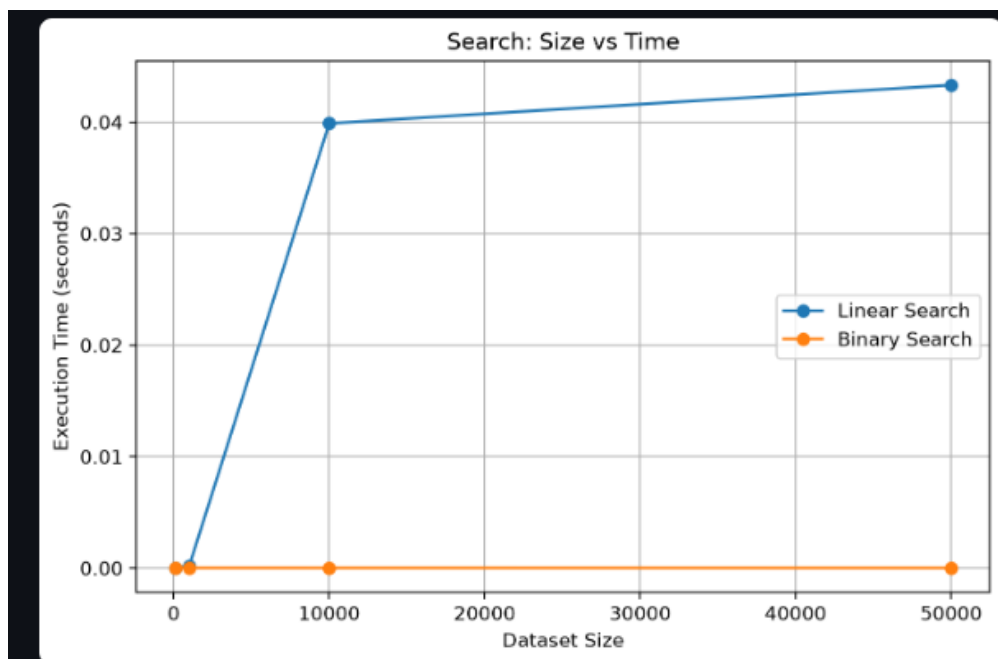


Figure 6: Search - dataset size vs execution time

Linear Search time goes up as the data size increases, but Binary Search stays almost at zero. This means Binary Search is much better for big data.

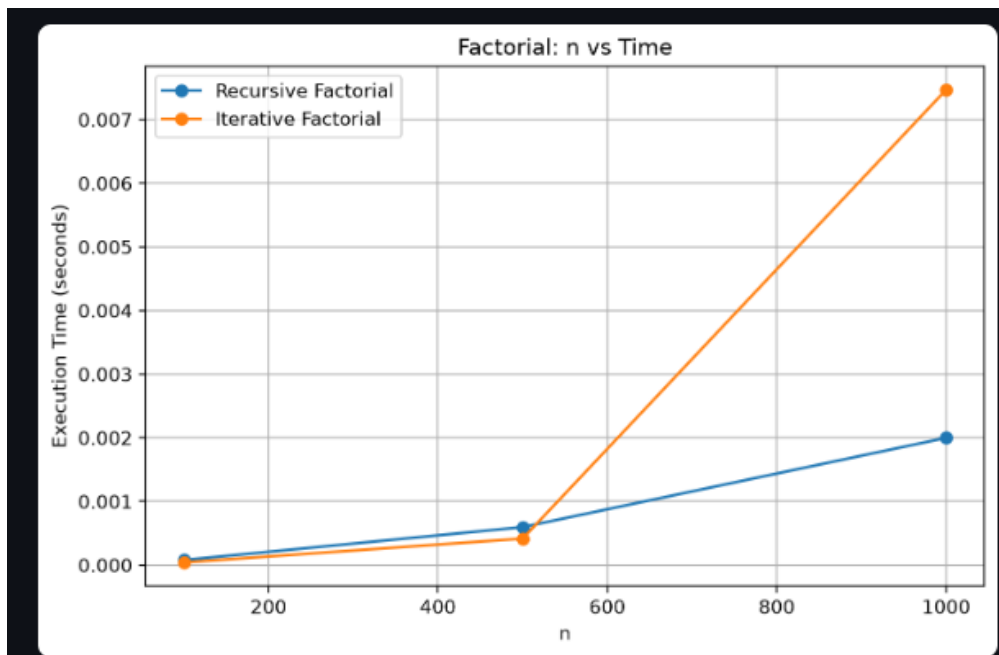


Figure 7: Factorial - input size vs execution time

Time of both factorial programs increases as n increases, which is the $O(n)$ growth. The two lines are close to each other, so there is no big difference in time between them.

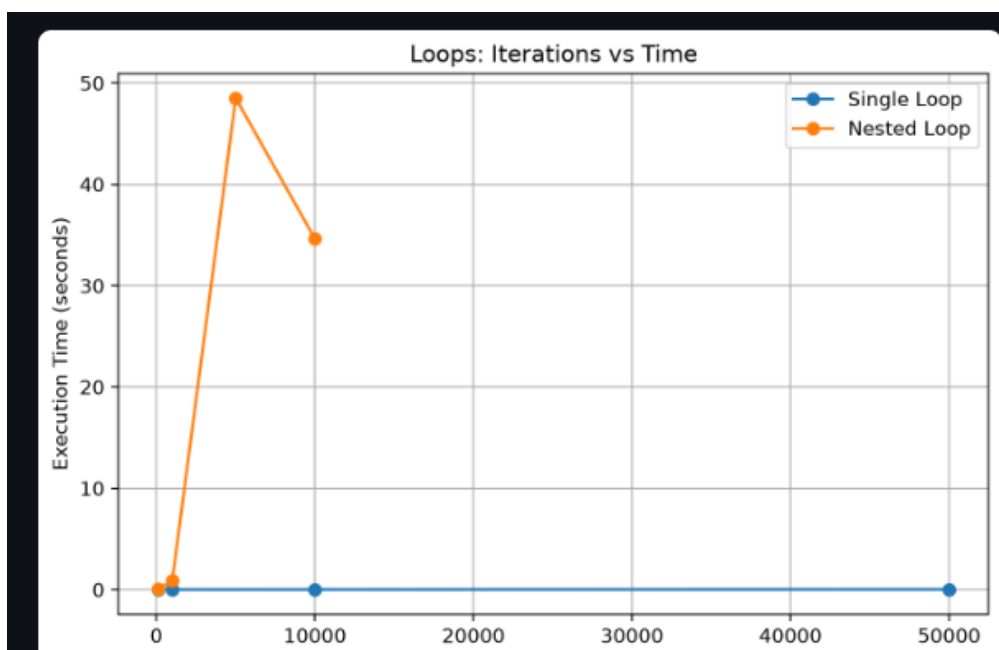


Figure 8: Loops - number of iterations vs execution time

Single Loop stays near zero even for 50,000 iterations, but Nested Loop takes a lot of time. The dip at $n = 10000$ is only because the computer was busy in the background; the nested loop should keep increasing.

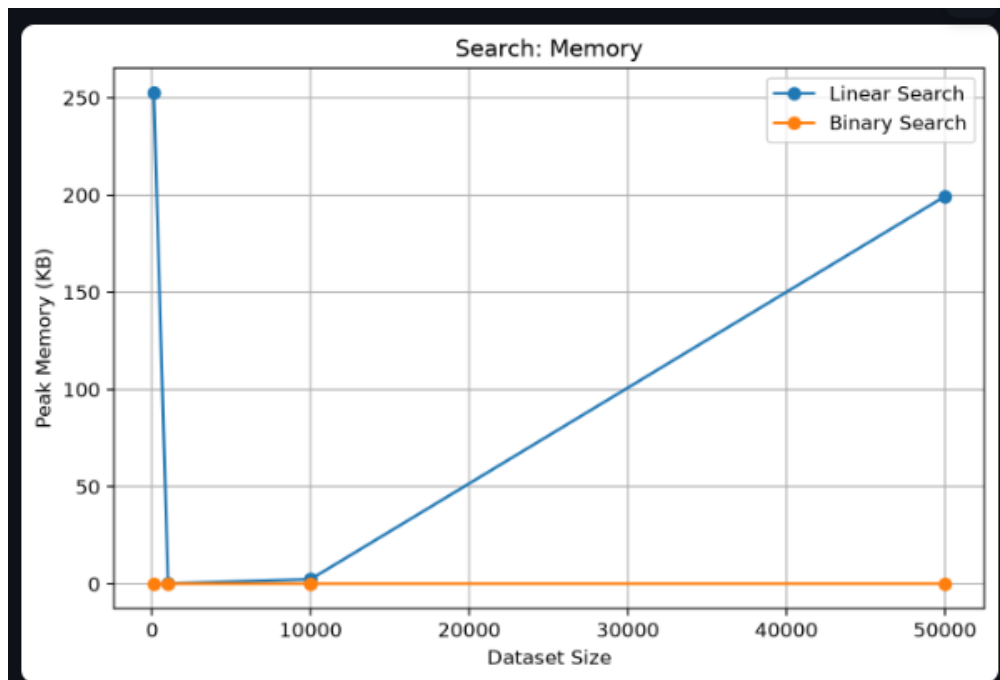


Figure 9: Search - dataset size vs peak memory

Binary Search uses almost no memory. The spikes in Linear Search are not from the algorithm, they come from background memory used by Python while measuring. Both searches need very little memory.

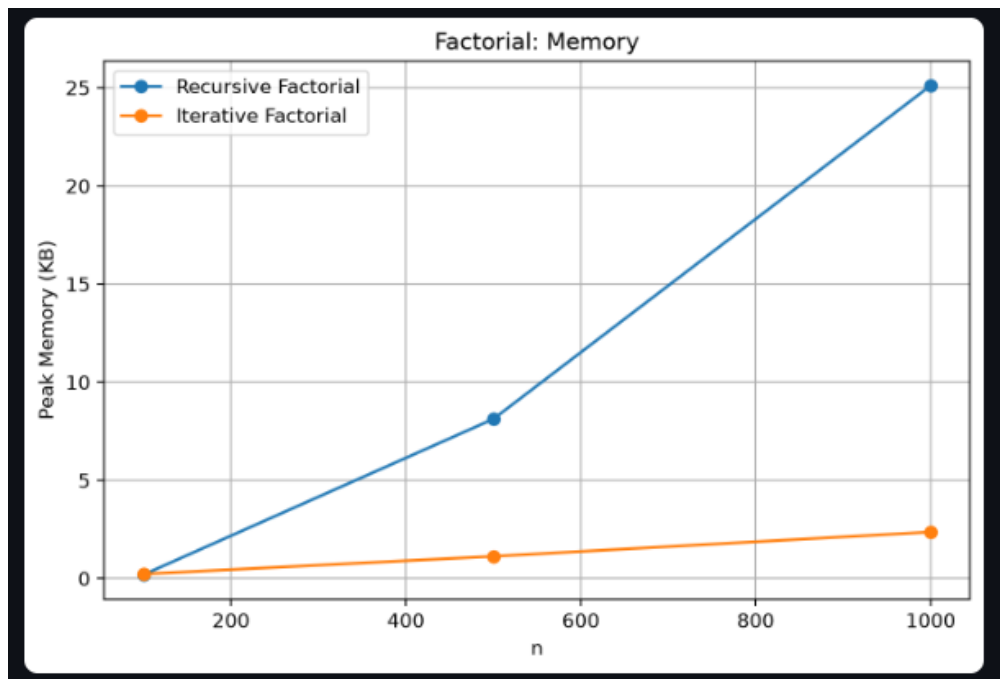


Figure 10: Factorial - input size vs peak memory

Recursive Factorial uses more and more memory as n increases because every function call is stored in the stack. Iterative Factorial uses very little memory. So iterative is better for memory.

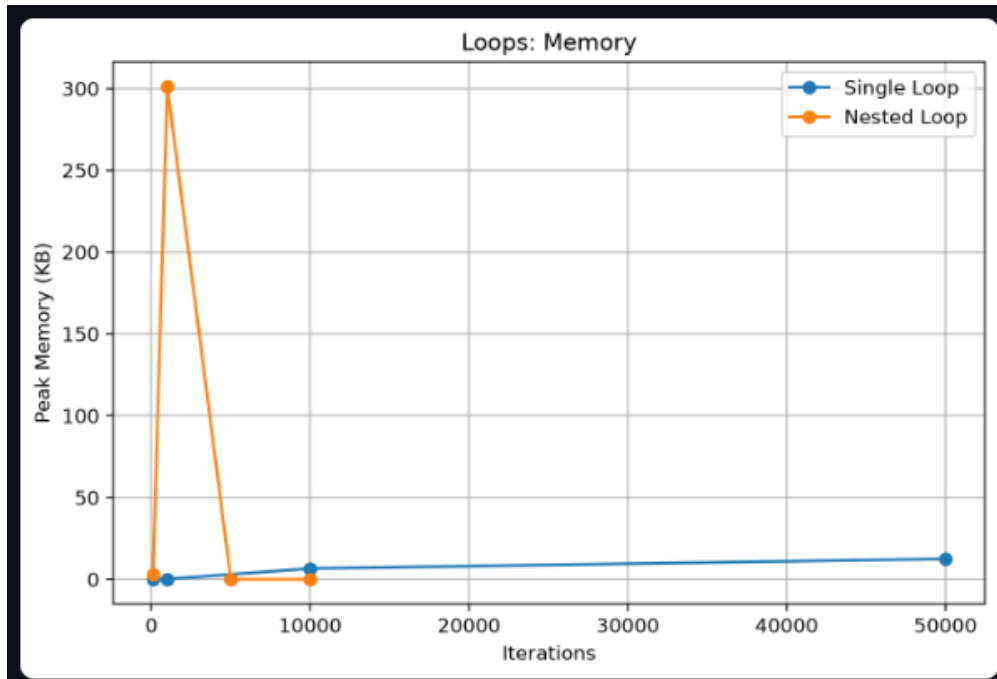


Figure 11: Loops - number of iterations vs peak memory

Memory of both loops is very small. The spike of the Nested Loop is not because of the loop, it is a measuring effect. So loops need very little memory and time is the main issue.

6.5 Complexity analysis output (Task 6)

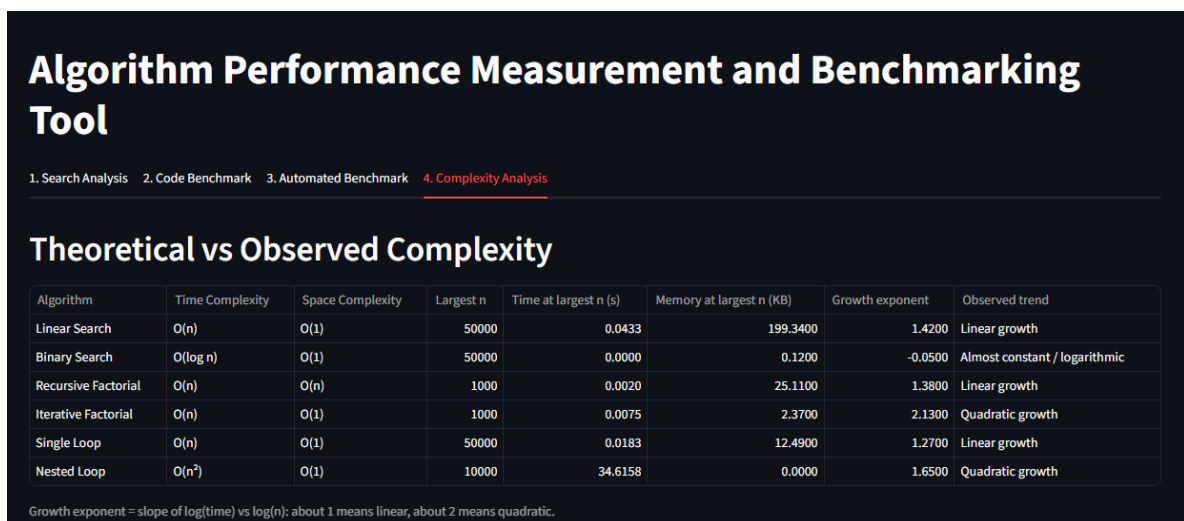


Figure 12: Theoretical vs observed complexity (Complexity Analysis tab)

This table compares theory with the observed results. Binary Search has growth exponent near 0 (almost constant), the linear programs are near 1, and Nested Loop is higher (1.65), moving towards quadratic. The values are not perfect because the computer was busy during the run.

6.6 Reference run from the project notebook

The notebook also saves `benchmark_results.csv` and `complexity_table.csv`. These saved results are from another run with less background load and are given here for comparison.

Automated Benchmark results

algorithm	input_size	time_sec	memory_kb	operations
Linear Search	100	0.000004	0.117	100
Linear Search	1000	0.000045	0.207	1000
Linear Search	10000	0.000491	1.611	10000
Linear Search	50000	0.002642	0.207	50000
Binary Search	100	0.000001	0.0	6
Binary Search	1000	0.000001	0.094	9
Binary Search	10000	0.000002	0.125	13
Binary Search	50000	0.000002	0.125	15
Recursive Factorial	100	0.000011	0.195	100
Recursive Factorial	500	0.000129	8.125	500
Recursive Factorial	1000	0.000340	23.75	1000
Iterative Factorial	100	0.000006	0.234	99
Iterative Factorial	500	0.000058	1.137	499
Iterative Factorial	1000	0.000198	2.375	999
Single Loop	100	0.000003	0.102	100
Single Loop	1000	0.000042	0.164	1000
Single Loop	10000	0.000459	0.164	10000
Single Loop	50000	0.002606	0.172	50000
Nested Loop	100	0.000418	0.188	10000
Nested Loop	1000	0.045421	0.281	1000000
Nested Loop	5000	1.304636	1.5	25000000
Nested Loop	10000	5.153218	2.0	100000000

Figure 13: Saved benchmark_results.csv (reference run)

Theoretical vs Observed Complexity

Algorithm	Time Complexity	Space Complexity	Largest n	Time at largest n (s)	Memory at largest n (KB)	Growth exponent	Observed trend
Linear Search	$O(n)$	$O(1)$	50000	0.002642	0.21	1.05	Linear growth
Binary Search	$O(\log n)$	$O(1)$	50000	2e-06	0.12	0.18	Almost constant / logarithmic
Recursive Factorial	$O(n)$	$O(n)$	1000	0.00034	23.75	1.49	Linear growth
Iterative Factorial	$O(n)$	$O(1)$	1000	0.000198	2.38	1.47	Linear growth
Single Loop	$O(n)$	$O(1)$	50000	0.002606	0.17	1.07	Linear growth
Nested Loop	$O(n^2)$	$O(1)$	10000	5.153218	2.0	2.05	Quadratic growth

Figure 14: Saved complexity table (reference run)

In this run the growth exponents are close to theory: Linear Search 1.05, Binary Search 0.18, Single Loop 1.07 and Nested Loop 2.05. Factorial is about 1.5 because the numbers become very big in $n!$. The nested loop for $n = 10000$ took about 5 seconds here.

Search Analysis output (10,000 elements)

Algorithm	Result	Comparisons	Time (s)	Memory (KB)
Linear Search	Found at index 7000	7001	0.00042746	0.207
Binary Search	Found at index 7000	10	0.00000155	0.156

Figure 15: Search output (reference run, 10,000 elements)

Code Benchmark output (n = 500)

Snippet	Time (s)	Memory (KB)	Operations
Single Loop Traversal	0.00002350	0.164	500
Nested Loop Traversal	0.01245268	0.281	250000
Recursive Factorial	0.00011991	8.125	500
Iterative Factorial	0.00005799	1.137	499

Figure 16: Code snippets output (reference run, n = 500)

7. Complexity Analysis: Theoretical vs Observed

The table combines the theoretical complexity with the operations counted and the growth exponents of the two runs.

Algorithm	Time	Space	Operations counted at largest n	Growth exp. (app run)	Growth exp. (reference run)	Observed trend
Linear Search	$O(n)$	$O(1)$	50000 ($n = 50000$)	1.42	1.05	Linear
Binary Search	$O(\log n)$	$O(1)$	15 ($n = 50000$)	-0.05	0.18	Almost constant
Recursive Factorial	$O(n)$	$O(n)$	1000 ($n = 1000$)	1.38	1.49	Linear (big numbers)
Iterative Factorial	$O(n)$	$O(1)$	999 ($n = 1000$)	2.13	1.47	Linear (big numbers)
Single Loop	$O(n)$	$O(1)$	50000 ($n = 50000$)	1.27	1.07	Linear
Nested Loop	$O(n^2)$	$O(1)$	100000000 ($n = 10000$)	1.65	2.05	Quadratic

The operation counts match the theory exactly: n for the linear programs, about $\log_2 n$ for Binary Search and n^2 for the nested loop. The time values follow the same order, with some noise on the computer.

8. Comparative Performance Study

8.1 Linear Search vs Binary Search

At 10,000 elements Linear Search made 6465 comparisons against 13 for Binary Search (Figure 1), and at 50,000 elements the worst case needed 50,000 comparisons against only 15. Binary Search is therefore much better for large sorted data. Its drawback is that the data must be sorted first. For a single search on unsorted data, sorting would cost more than one linear scan, but when many searches are done on the same data, Binary Search is far more efficient. Both use $O(1)$ extra space.

8.2 Recursive Factorial vs Iterative Factorial

Both have $O(n)$ time and perform about n multiplications. The clear difference is space: at $n = 1000$ the recursive version used about 25 KB and the iterative version about 2.4 KB (Figure 10). Recursion also has a depth limit in Python (we had to raise it to 5000 for $n = 1000$), while the loop has no such limit. The measured times of the two were very close and the small differences changed between runs, so neither can be called faster on time alone. The iterative method is the safer choice because of its $O(1)$ space.

8.3 Single Loop vs Nested Loop

The Single Loop did 50,000 operations in a few milliseconds, while the Nested Loop did 100 million operations for $n = 10000$ and needed several seconds (5 s in the reference run). When n increases 10 times, a single loop needs about 10 times more steps, but a nested loop needs 100 times more steps. This shows why $O(n^2)$ programs become impractical for large inputs.

8.4 Relationship between complexity and actual time

Complexity predicts how the time grows, not the exact time. The operation counts matched the theory perfectly. The measured times follow the same trend, but small values (microseconds) change from run to run and are affected by the other programs running on the computer. This is why several repetitions are averaged and why operation counts are a useful second measure.

8.5 Time-space trade-offs

Implementation	Time	Space	Comment
Recursive Factorial	$O(n)$	$O(n)$	Short and natural code, but uses stack memory and has a recursion limit
Iterative Factorial	$O(n)$	$O(1)$	Same time, much less memory, no recursion limit
Binary Search	$O(\log n)$	$O(1)$	Very fast, but needs the data to be sorted first
Linear Search	$O(n)$	$O(1)$	Simple, works on unsorted data, slow for large data

9. Performance Observations and Limitations

- Very small times (microseconds) change between runs, so the average of several runs is used.
- The application run was done while Streamlit, the browser and OneDrive (syncing the venv folder) were working in the background. This explains the higher times for Linear Search and the nested loop, and the unusual point of the nested loop at $n = 5000$ which is slower than $n = 10000$.
- The nested loop is run only once because it is slow; a single run is more affected by disturbance than an average of five runs.
- Memory values from tracemalloc for small programs are very small and can show spikes (for example 252 KB for Linear Search at $n = 100$) because Python allocates memory in the background, especially in the first run.
- The nested loop with $n = 50,000$ was not run because it needs 2.5 billion steps and would take many minutes in Python; sizes up to 10,000 were used.
- Absolute times depend on the computer, but the growth trends and the operation counts stay the same.